

Java Generics

Java Generics permite que tus clases base sean reutilizables. Creas una plantilla de código que puede funcionar con cualquier tipo de dato, como Cliente o Producto.

Se usan símbolos como <T> para representar un tipo genérico. Al usar la clase base, especificas el tipo concreto que necesitas.

Controlador Base Genérico

```
// Este controlador funciona para cualquier entidad 'T'.
public abstract class ControladorBase<T, ID> {

    protected ServicioBase<T, ID> servicio;

    // Endpoint genérico para obtener por ID.
    @GetMapping("/{id}")
    public ResponseEntity<?> obtenerPorId(@PathVariable ID id) {
        Optional<T> entidad = servicio.obtenerPorId(id);
        if (entidad.isPresent()) {
            return ResponseEntity.ok(entidad.get());
        }
        return ResponseEntity.notFound().build();
    }
}
```

Controlador Específico

```
@RestController
@RequestMapping("/api/clientes")
public class ClienteController extends ControladorBase<Cliente, Long> {
    // La lógica principal se hereda, no es necesario reescribirla.
}
```

El beneficio principal es que escribes la lógica común una sola vez. Esto reduce la duplicación de código.

Java Reflection

Java Reflection es una técnica que permite al código analizarse y modificarse a sí mismo

mientras se ejecuta.

Su uso común en este contexto es automatizar tareas como el mapeo de objetos, por ejemplo, para convertir un DTO en una Entidad sin escribir el código para cada campo.

Se debe usar con precaución por las siguientes razones:

- Rendimiento: Es más lento que el código normal y puede afectar el rendimiento.
- Seguridad: Se pierde la verificación de tipos en la compilación. Los errores aparecen en tiempo de ejecución.
- Complejidad: El código se vuelve más difícil de leer y mantener.

Recomendación

Para el mapeo de objetos, es preferible usar librerías como MapStruct o ModelMapper. Son más eficientes y seguras. Reflection se reserva para casos muy específicos donde la flexibilidad dinámica es un requisito fundamental.